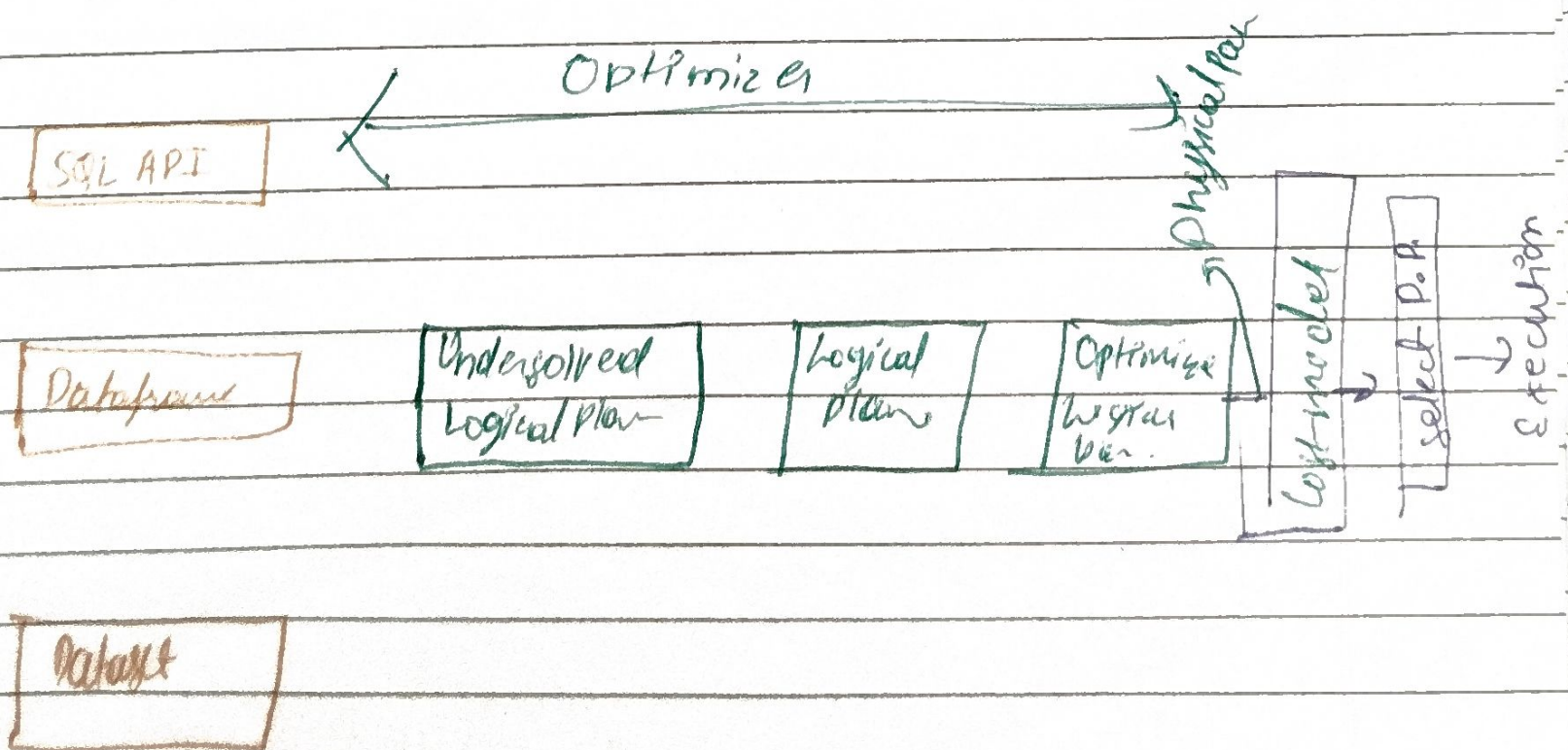


9. Catalyst Optimizer

- Scala program provided by Spark for Dataframe / SQL API, that automatically find out the most efficient plan to execute data operation specified in user code.
- Rule based Optimizer :- Set of predefined rules is used to design the logical plan.
- Cost-based optimizer :- Based on available statistics collection of underlying data, cost is estimated and best efficient execution approach is decided based on cost.



10. Compression Methods

Snappy Vs Gzip

* Snappy

- Low CPU Utilization
- Low Compression Rate (large compress file)
- Splittable :- Parallel processing
- Use case :- Hot Layer
- Compute Intensive :- when compute is important

* Gzip

- High CPU Utilization
- Highly Compressed file (small file)
- Non-Splittable
- Use case - Cold layer
- Storage is important

```
• df.write.format("parquet").option("compression", "snappy").  
  save("path")
```

```
• df.write.format("parquet").option("compression", "gzip").  
  save("path")
```


11. Optimized Autoscaling

StarLine
Date: / /
Page:

- Databricks chooses dynamically the appropriate number of workers required to run the job based on range of number of workers.
- It is also one of the cost saving technique.

① Optimized
② Standard

There are two types of clusters

- ① Job cluster (Automated) → Optimize Auto scaling
- ② General purpose (All purpose)

Standard Autoscaling

1. Scale based on CPU load
2. Add/Remove worker nodes (slowly, 10min)

•

- It stable, but not cost efficient

Optimized Autoscaling

- It uses Spark execution plan + runtime metrics
- Add and remove nodes quickly (40 sec)

12. UDF

User Defined Function is piece of code which perform certain task and can be reused to perform the same task across multiple scenarios

- UDF is black-box for spark engine.
- UDF is python object not understood by JVM and dataframe is your object.

```
def rename_col(rename_df):
```

still about

```
    for column in rename_df.columns:
```

```
        new_col = "col_" + column
```

```
        rename_df = rename_df.withColumnRenamed(
            column, new_col)
```

```
    return renamed_df
```

```
def add_one(x):
```

```
    return x+1
```

off.

13. Delta Table : Optimize

A ~~delta~~ delta table is a storage format built on top of parquet that provide ACID transaction and reliability for data lake.

It store

- Data in Parquet file
- Metadata and history in a - delta log folder
- It support, Insert, Update, Delete, Merge
- Provide time travel
- Ensure data consistency (ACID)
- Handle Schema ~~en~~ enforcement
- Delta makes a data lake behave like a database.

OPTIMIZE Command

The Optimize command is used to improve query performance by ~~compact~~ compacting small files into large files.

Why needed :

Spark write many small files → slow query operation.

- Optimize combine files into fewer large files.
- Reduce file scanning overhead
- Improve read performance

OPTIMIZE table_name;

OPTIMIZE table_name ZORDER BY ("Age")

Optimize : Deleted

- Optimize does not delete data.
- Time Travel still work after Optimize
- Old files are removed only after ~~Vac~~ Vacuum.

14: Vacuum Command

Vacuum Command is used to permanently delete old and ~~uns~~ unused data files from a Delta table:

When data is updated, ~~data~~ delete or optimize:

- Old files are not removed immediately
- They remain for time travel and recovery

→ Vacuum removes these old files.

~~Vacuum Command~~

~~Vacuum~~

VACUUM table_name:

VACUUM table_name RETAIN 168 HOURS.

default retention = 7 days.

- After Vacuum ~~to~~ time travel ~~&~~ not work for old ~~o~~ version

Vacuum table_name RETAIN 168 HOURS

DRY RUN.

15. Z-Ordering

• In delta lake, it re-orders data inside file based on one or more column to improve data skipping while we perform optimize command.

- It solve problem of variable range (insistent range of table column)

•

OPTIMIZE table-name ZORDER BY (column)

16. Sort Merge Join

(SMJ)

In this join data is shuffled on join key, then sorted within ~~part~~ partitions and then merged to produce the result.

Why it exist :-

- Same join key are spread across machine

17. Bucketing

- Bucketing is a data organization technique in Spark that pre-shuffles and pre-sorts data into fixed number of files (buckets) based on a hash of a key column, then store in persistently.

- How it works:

1. Data is hashed using a specified column
2. Hash result is modulo'd by number of buckets
3. Rows with same hash go to same bucket

- Why Bucketing Exist

1. Optimized Joins:- Pre bucketing tables on join key. =
Matching rows in same bucket.
→ Result in faster join.

2. Efficient Aggregation:- Group By on bucket column
= data already organized.

3.

- Only go for bucket by when the $\Rightarrow$ high cardinality

18. Delta Cache

- Spark cache is in memory persist
- Delta cache is disk persist

DRIO, DELTA cache and Disk cache
are the same name

When delta table is stored in external remote location the data need to be fetched from the them.

Efficient decomposition

```
spark.conf.set("spark.databricks.io.cache.enabled", true)
```

for pre-loading

Cache Select * from table;

19. Select vs WithColumn

- WithColumn :- WithColumn used to add or replace a single column of an existing df.
- Select :- Primarily used to select columns, but can also add new computed columns on the ~~top~~ fly.

The core problem :-

PySpark dataframes are immutable by design. Every time you call withColumn, Spark internally creates a brand new dataframe.

What with column does

~~can~~ take a df and add new column

→ new dataframe

take new dataframe add new column

→ new dataframe

It keep creating new ~~a~~ dataframe as apply withColumn

But select

add ~~g~~ directly those computed columns on fly.

..

- WithColumn create a new internal DataFrame on every call leading to deeply nested logical plans and repeated shuffled operation.
- Select combines all columns additions into a single plan with a ~~for~~ flat operation.

32
↓

StarLine

Date: / /

Page: .

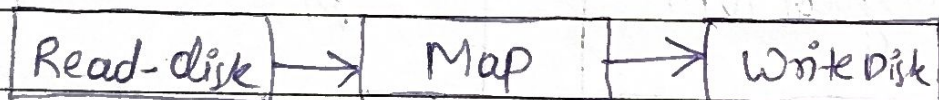
20. Internals of Partitions

1. Spark Architecture

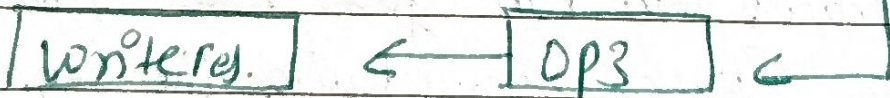
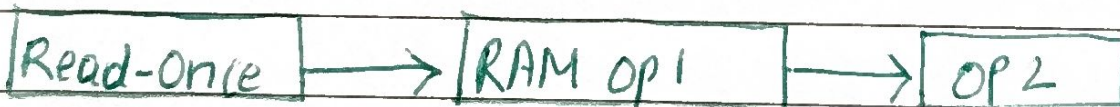
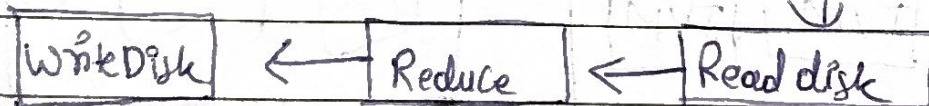
Spark is a unified, open-source distributed computing engine designed for large-scale data processing.

Why Spark is Fast?

1. In-memory processing :- Faster than disk & faster than Hadoop
2. Parallel Processing :- Suppose that 1 TB of data needs to be processed, A 1 TB dataset split into 10,000 partitions across 100 machines processing 100 partitions in parallel at once.



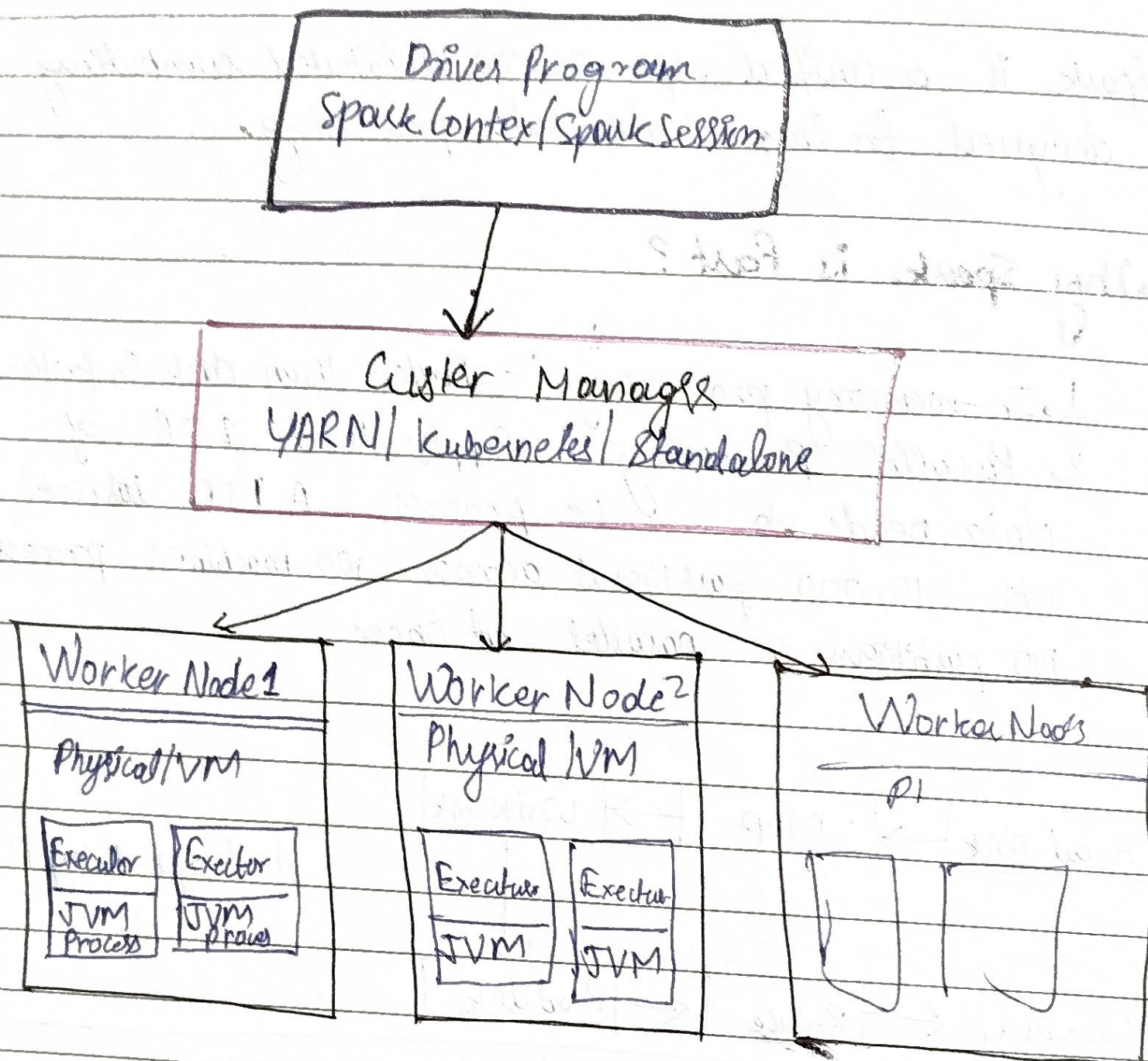
Hadoop MapReduce



* High-Level Spark Architecture

Master-worker architecture, There are three main components.

1. Driver
2. Node Manager / Cluster Manager
3. Worker Node



• **Driver** :- The process that runs your main functions. It creates the Spark Session, builds the logical plan, schedules tasks and tracks their execution, optimizes the plan.

• **Cluster Manager** :- An external service that negotiates resources (CPU cores, memory) from the cluster on behalf of the driver. Spark supports YARN, Kubernetes, Mesos, and its own Standalone manager.

- **Worker Node** :- Physical or virtual machine that host Executors. They receive task from the driver and report result back.

Internal Structure of Worker Node

A worker node is a machine. Inside it, Spark launches one or more Executors — each an independent JVM process.

- **Executor** :- A JVM process on a worker Node. Each application get its own set of Executors that live for the application lifetime.

- **Cores** Each Core = one parallel task slot. An Executor with 8 Cores run 8 tasks simultaneously.

- **Partition** : Data is split into partitions. Each task processes exactly one partition. More partition = More parallelism (up to number of available Core)

- **Memory** :- Split into Executor memory (for shuffle, join) and Storage Memory (for caching/persisting RDDs and dataframe)

• Key Attributes

1. Scalability:- Spark scales horizontally. Add more nodes, get more parallelism. It routinely runs on ~~etc~~ clusters of thousands processing petabytes.
2. Lazy Evaluation:- Transformations are not executed immediately. Spark builds up a logical plan (DAG) and only executes when an Action is called.

This allows the optimizer to take as many as many operations, rearrange and combine operation.

3. Polyglot:- Spark supports Scala (primary), Java, Python, R, SQL.

4. ~~Spark~~ Streaming Capabilities:- Spark Streaming and Structured Streaming allow processing of real-time data streams using the same DataFrame API as batch.

5. Fault Tolerance:- If an Executor fails, Spark recomputes only the lost partition using the DAG lineage. It knows exactly how to rebuild any piece of data.

• Core Spark Terminologies

- **Application :-** Your Spark program. The program that starts Spark has both driver and all its ~~executors~~ executors.
- **Job :-** Created each time an Action is called.
- **Stage :-** A job is divided into stages at shuffle boundaries.
 • So a stage is set of operations that can run without requiring a ~~sh~~ shuffle.
- **Task :-** Task is smallest unit of work.
 One task = One partition
 So Number of tasks in a stage = number of partitions in that stage.
- **Driver :-** Orchestrates everything - It's a program ~~not~~ not a machine that manages all.
- **Workers :-** A run in the cluster that runs Executors.

• Transformation Vs Action

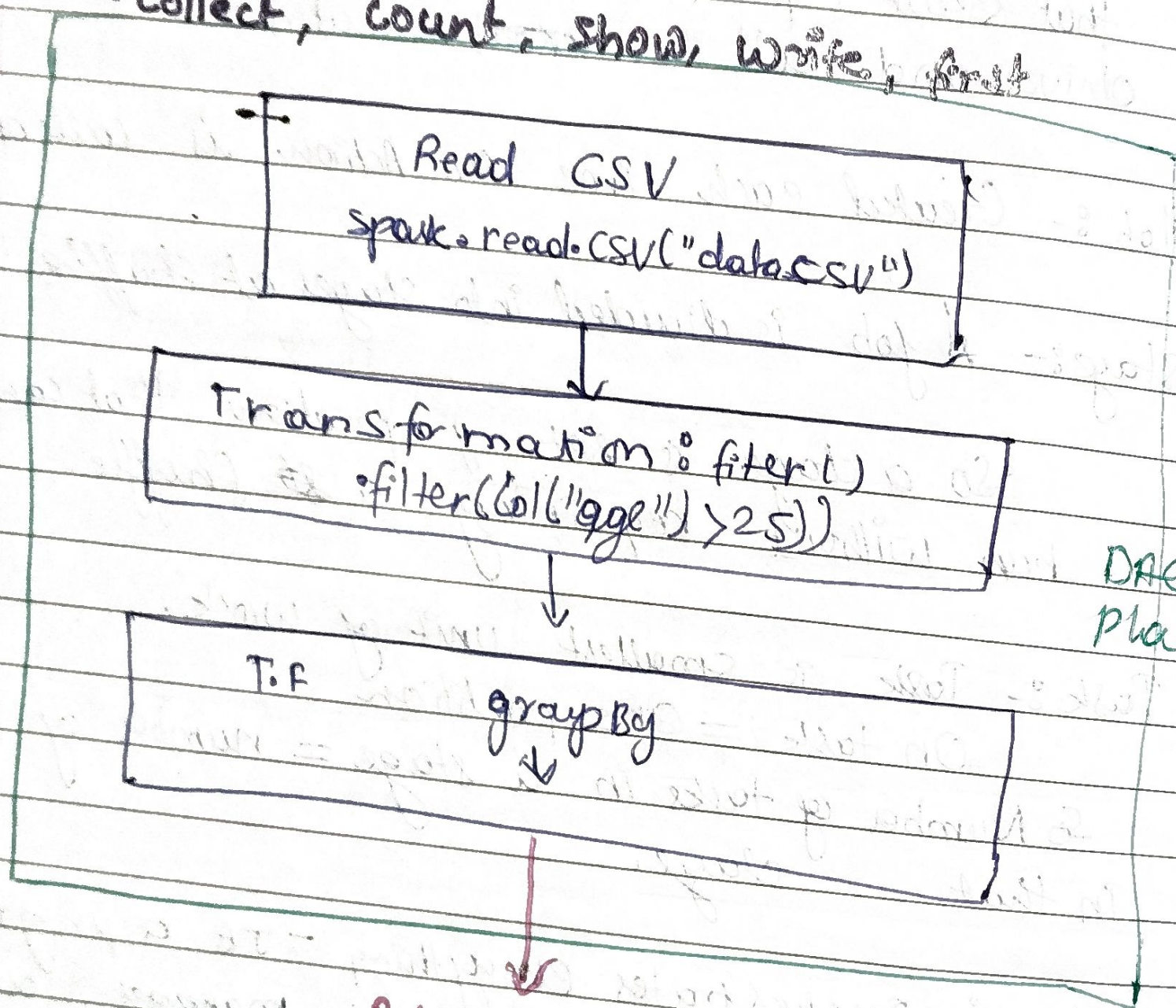
Operations on DataFrames/RDD are divided into two categories.

- **Transformations are Lazy :-** They define what to do but don't execute anything. They build up

the DAG. e.g. Filter, groupBy, join, map, select

- Actions trigger execution of the entire accumulated DAG.

- collect, count, show, write, first



Action - show()

Entire DAG executes Now after optimization

• Spark Data Structures and Concepts •

• ~~RDD~~ RDD (Resilient Distributed Dataset)

The original, low-level abstraction. It's an immutable distributed collection of objects. It offers full control but no automatic optimization. It's a raw Scala object.

• **DataFrame**:- A distributed table with named, typed columns. Backed by Spark's Catalyst Optimizer.

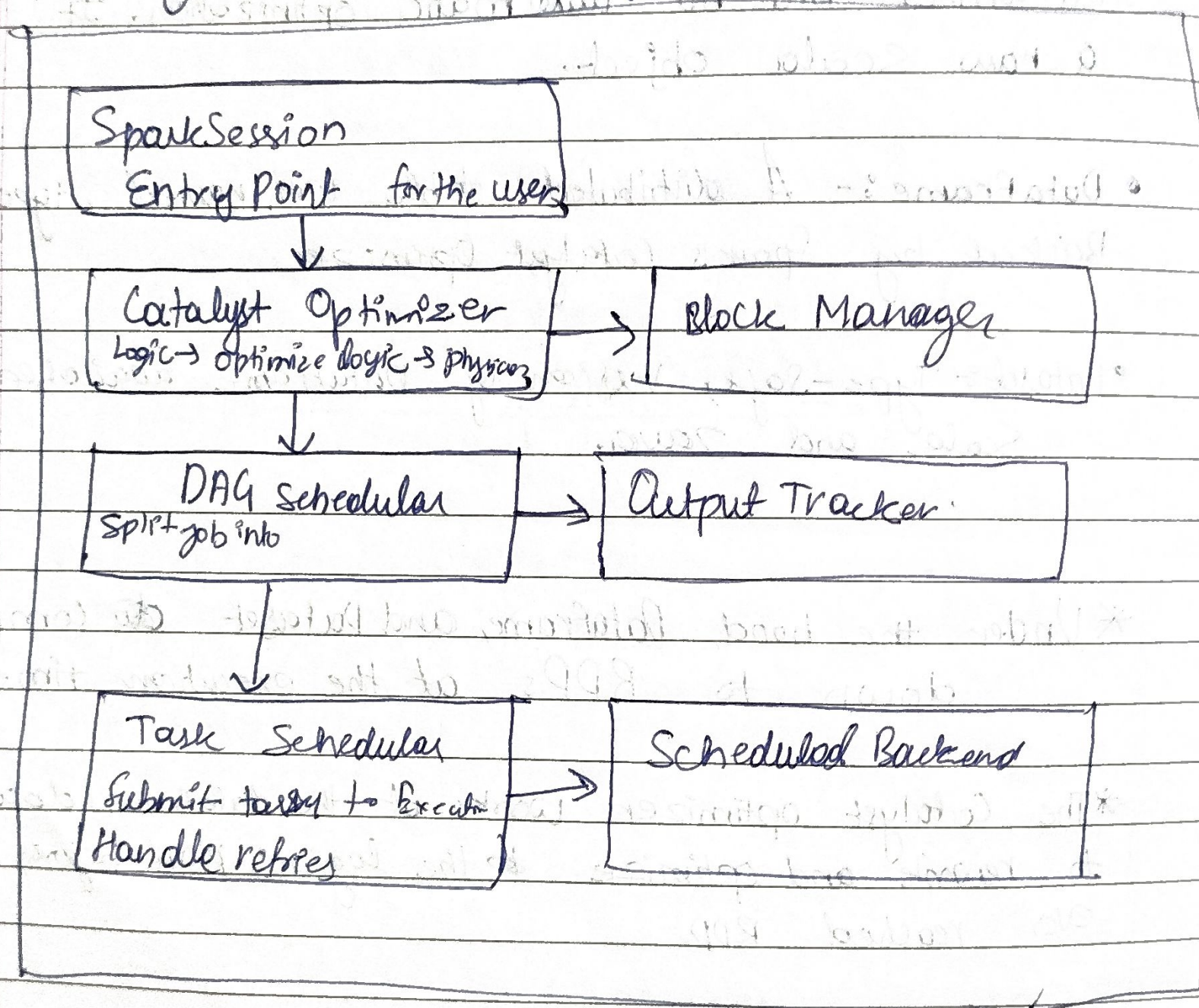
• **Dataset**:- Type-Safe, Version of DataFrame available in Scala and Java.

* Under the hood, DataFrame and Dataset compile down to RDDs at the execution time.

* The Catalyst optimizer works at the DataFrame/dataset to rewrite and optimize the logical plan before it even reaches RDD.

• Internal Drives Architecture

The Drives is just not "the main program" — it's several internal components that form the planning and scheduling.



• Catalyst Optimizer :- DF operations → logical Plan and applies rule based and cost based optimization.

• DAG Scheduler :- Takes the physical plan and divide it into stage based shuffle dependencies.

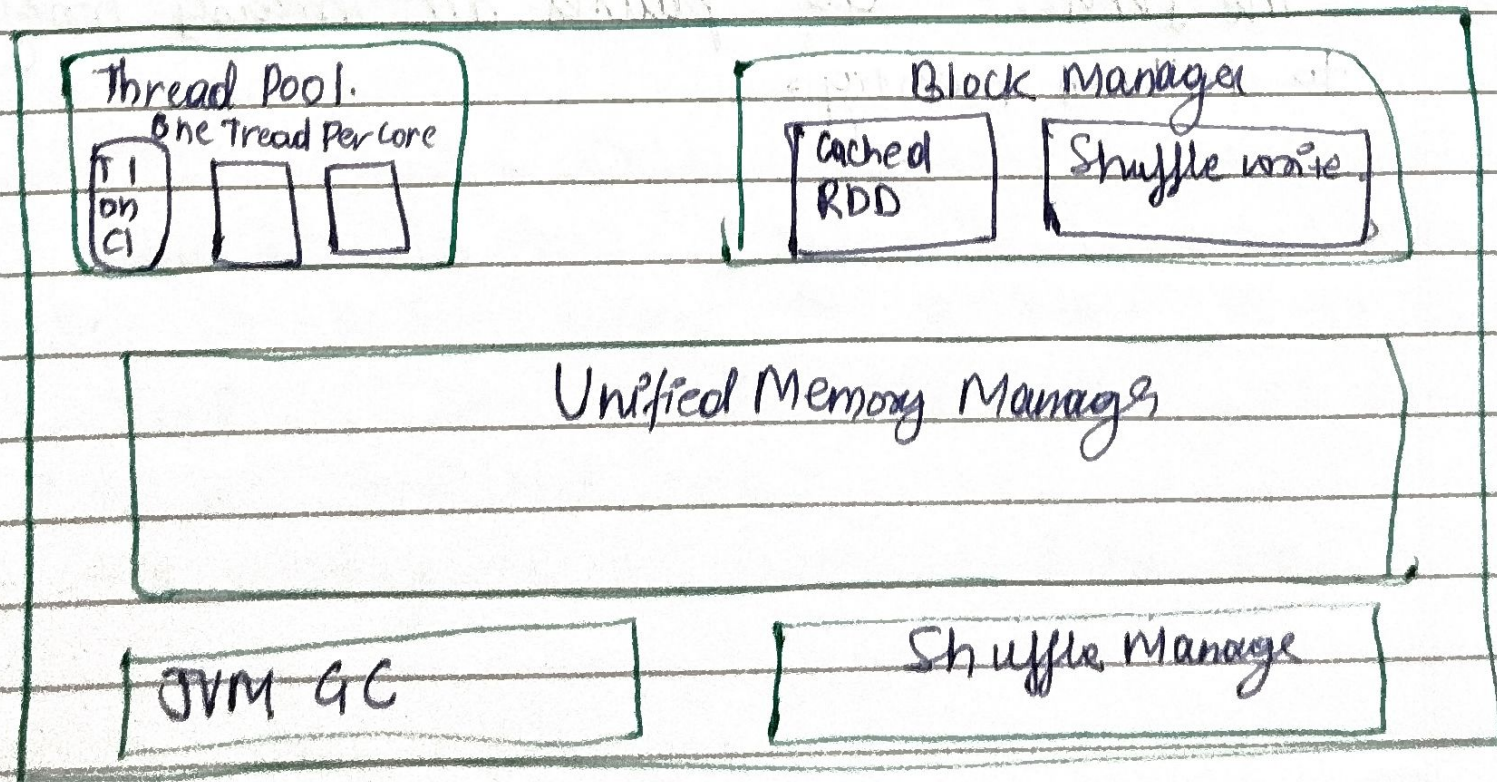
It build the DAG of stage dependencies and submit ready stage to the Task Scheduler.

• Task Scheduler:- ~~Task~~ Takes a stage's tasks and assign them executor slot. It handles locality ~~prop~~ preference. Retries tasks that fail.

• Block Manager:- A distributed storage system inside Spark for caching RDD partitions, broadcast variable and shuffled block.

* Worker Node Architecture.

Inside each executor, there are several internal components that manages execution.



• **Tread Pool:-** Each core in the Executor runs one task thread.

• **Block Manager (local) :-** Store & cached RDP partition, broadcast Variable. And it communicate with the Driver's master Block Manager for Coordination.

• **Shuffle Manager :-** When a wide transformation occurs the shuffle manager on the write side of write intermediate output file partitioned by key. On the read side, it fetches those files over the network.

• **JVM Garbage Collector :-**
The JVM GC pauses all threads briefly to reclaim memory.

21. Delta Table

Column Reordering

- In traditional DB, for reordering column in table
~~we first~~ the engine first take backup → drop table → Recreate the new table with ordered column → Reload → drop the backup.

- But in delta table, it just meta data reordering. Since data is store in columns form and its non-contiguous. So it very efficient to reorder column in delta table.

~~Delta Table~~

* The performance reason

- Delta Store

1. Data → parquet

2. Metadata → - delta.log (JSON log)

- **Stats collected only for first 32 columns**

* So when we perform query on a column if the stats exist for that column then we can use it to skip irrelevant files.

- If stats missing → full scan of all file.

Problem

→ If frequently used columns not in first 32 this will result in

→ No Stats

→ Full Scan

→ Performance hit

Kirti Man Bhai

• Solution :-

Move frequently filtered columns → with first 32 columns

Alter Table table_name

CHANGE COLUMN column_name Column_name
Datatype First

• Reordering only affects :- only future writes

Old data:- Still has stats for old column order and
Optimization not applied

To solve this.

ANALYZE TABLE table_name
COMPUTE STATISTICS FOR ALL Column

22. Deletion Vector

• Problem with Standard Delete

1. Delete / Update / Merge :

→ Rewrite entire parquet files

→ Even a small change → full file rewrite

• Impact

1 :- High I/O cost

2 :- Increased storage cost

3. Slow write/update.

• What is deletion vector

• Instead of rewriting file :- Store row-level delete markers that tell which row has been deleted.

The delete vector is a ~~bin~~ binary index file (.bin)

• How Deletion Vectors work

• Original Parquet file → unchanged

• Deleted rows → marked in .bin file

• Now at the time of read operation

• Read delta log + Read parquet data

• Filter out the deleted rows after reading .bin

• Key Benefit

→ Massive improvement in writing performance

→ Reduce storage overhead

→ Slightly add complexity while reading.

Syntax

```
CREATE TABLE table_name  
TBLPROPERTIES ('delta.enableDeletionVectors' = true);
```

```
ALTER TABLE table_name  
SET TBLPROPERTIES ('delta.enableDeletionVector' = true);
```